THE MINISTRY OF SCIENCE AND HIGHER EDUCATION
OF THE RUSSIAN FEDERATION

ITMO University
(ITMO)

Faculty of Control Systems and Robotics

Task 1 (Variant 2)
for the subject
"Simulation of Robotic Systems"

Student:
Group No. R4137c *Mughal, Umer Ahmed Baig*
ITMO ID. 472268

Tutor:
Professor *Borisov, Ivan*

Assistant *Rakshin, Egor*

Saint Petersburg 2024

CONTENTS

0.1 Objective 1
0.2 Steps to Perform 1
0.3 Variant 1

1 ODE OF 2ND ORDER 2
1.1 Mass Spring System 2
1.1.1 Kinetic Energy (K) 2
1.1.2 Potential Energy (P) 3
1.2 Lagrangian Equation 3
1.3 Euler-Lagrange Equation 3
1.4 Full Equation of Motion 4

2 SIMULATION 5
2.1 Defining the State 5
2.2 Defining Integrators 5
2.2.1 Explicit Euler 5
2.2.2 Implicit Euler 6
2.2.3 Runge-Kutta of order 4 (RK4) 8
2.3 Comparison of Results 9
2.3.1 Comparison of Integrators of Mass Spring System 10

3 CONCLUSION 12

TASK DESCRIPTION

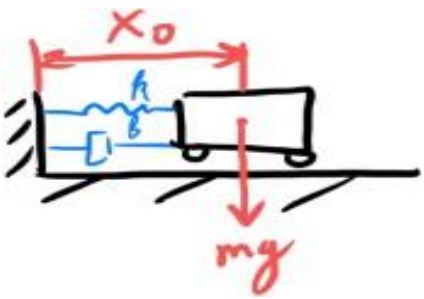
0.1 Objective:

To perform analytical and simulation modeling of 1D system according to the variant.

0.2 Steps to perform:

1. Compose the ODE of the 2nd order of the system.
2. Write out the function mass_spring_system according to the example of the script in lecture 1.
3. Compose three integrators based on explicit Euler, Implicit Euler and Runge-Kutta method of order 4.
4. Compare simulation results, draw conclusions.

0.3 Variant:



(a)

Figure 1: a - A mass-spring system with a mass, damping coefficient, spring constant, and the effects of gravity.

m, kg	k, N/m	b, N.s/m	X_0, m
0.4	4.4	0.025	0.46

(b)

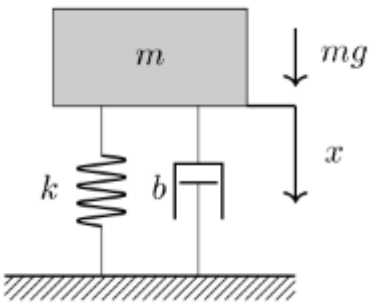
Figure 1: b - Given values according to the variant.

1 ODE OF 2ND ORDER

To compose the second-order ordinary differential equation (ODE) for the given system we will use Lagrangian method. The Lagrange's Equations have the advantage of requiring only the kinetic and potential energies of the system to be computed.

1.1 Mass Spring System

Consider a mass m attached to a spring with a spring constant **k** and a damping coefficient **b**. The mass moves along a single axis, with $\boldsymbol{x(t)}$ representing the displacement of the mass from its equilibrium position. Gravitational Force **mg** acting downward, which we assume is balanced at equilibrium and does not affect the oscillatory motion in $\boldsymbol{x(t)}$.



The Lagrangian L is defined as the difference between the kinetic energy K and the potential energy U of the system:

$$L = K - U \tag{1}$$

1.1.1 Kinetic Energy (K)

The kinetic energy of the mass is related to its horizontal velocity, as it moves along the horizontal axis with displacement $\boldsymbol{x(t)}$ and is given by:

$$K = \frac{1}{2} m \dot{x}^2 \tag{2}$$

Where: $\dot{x} = \frac{dx}{dt}$ is the velocity of the mass.

1.1.2 Potential Energy (U)

The potential energy stored in the spring (assuming it follows Hooke's law) and is given by:

$$U = \frac{1}{2}kx^2 \tag{3}$$

1.2 Lagrangian Equation

The Lagrangian for the system is the difference between kinectic energy and potential energy: Substitute K and U into the Lagrangian equation:

$$L = K - U$$

$$L = \frac{1}{2}m\dot{x}^2 - \frac{1}{2}kx^2 \tag{4}$$

1.3 Euler-Lagrange Equation

To drive the equation of motion, we use Euler-Lagrange's equation:

$$\frac{d}{dt}\left(\frac{\partial L}{\partial \dot{x}}\right) - \frac{\partial L}{\partial x} = 0 \tag{5}$$

We need to compute $\frac{\partial L}{\partial \dot{x}}$, $\frac{d}{dt}\left(\frac{\partial L}{\partial \dot{x}}\right)$, and $\frac{\partial L}{\partial x}$.

Calculating $\frac{\partial L}{\partial \dot{x}}$

$$\frac{\partial L}{\partial \dot{x}} = m\dot{x}$$

Now calculate $\frac{d}{dt}\left(\frac{\partial L}{\partial \dot{x}}\right)$

Taking the time derivate:

$$\frac{d}{dt}\left(\frac{\partial L}{\partial \dot{x}}\right) = \frac{d}{dt}(m\dot{x}) = m\ddot{x} \tag{6}$$

Calculate $\frac{\partial L}{\partial x}$.

$$\frac{\partial L}{\partial x} = -kx \tag{7}$$

Finally, substituting these expressions into the Euler-Lagrange equation:

$$m\ddot{x} - (-kx) = 0$$

or

$$m\ddot{x} + kx = 0 \tag{8}$$

In our system, the damping force is given by:

$$F_d = -b\dot{x}$$

Where **b** is the damping coefficient. Since this force is non-conservative, we add it directly to the equation of motion.

1.4 Full Equation of Motion

The equation of motion for the mass-spring system, including the damping coefficient, is:

$$m\ddot{x} + b\dot{x} + kx = 0 \tag{9}$$

Dividing by m, we can rewrite the equation as:

$$\ddot{x} + \frac{b}{m}\dot{x} + \frac{k}{m}x = 0 \tag{10}$$

2 SIMULATION

2.1 Defining the State

In Google Colab, we defined the mass spring system as follows:

```python
# Mass Spring System
def mass_spring_system(state: list):

    # x is position and dx is velocity
    x, dx = state

    # System parameters
    b = 0.025  # damping coefficient in N.s/m
    m = 0.4    # mass in kg
    g = 9.8    # gravitational acceleration (m/s^2)
    k = 4.4    # spring constant in N/m

    # Equation: ddx = -1/m * (b*dx + k*x)
    ddx = -1/m * (b*dx + k*x)

    # Return derivatives as an array
    return np.array([dx, ddx])
```

Figure 2: State function

2.2 Defining Integrators

2.2.1 Explicit Euler

This approach, known as the Forward Euler method, approximates the next state by using the current state and its derivative.

$$\text{state}_{n+1} = \text{state}_n + \Delta t \cdot \text{mass-spring system}(\text{state}_n) \tag{11}$$

We define this function in Google Colab and run the simulation with a time step of 0.01.

```python
# Forward Euler method for Mass Spring System
def forward_euler(fun: object, x0: np.ndarray, t_f: float, h: float) -> tuple:
    # Create a time array from 0 to final time t_f, with step size h
    t = np.arange(0, t_f + h, h)
    x_hist = np.zeros((len(x0), len(t)))
    x_hist[:, 0] = x0  # Set the initial state

    for k in range(len(t) - 1):
        x_hist[:, k+1] = x_hist[:, k] + h * fun(x_hist[:, k])

    return x_hist, t
```

Figure 3: Defining the Explicit Euler Integrator

By running the simulation, we get the following graph illustrating the state of the system with respect to time.

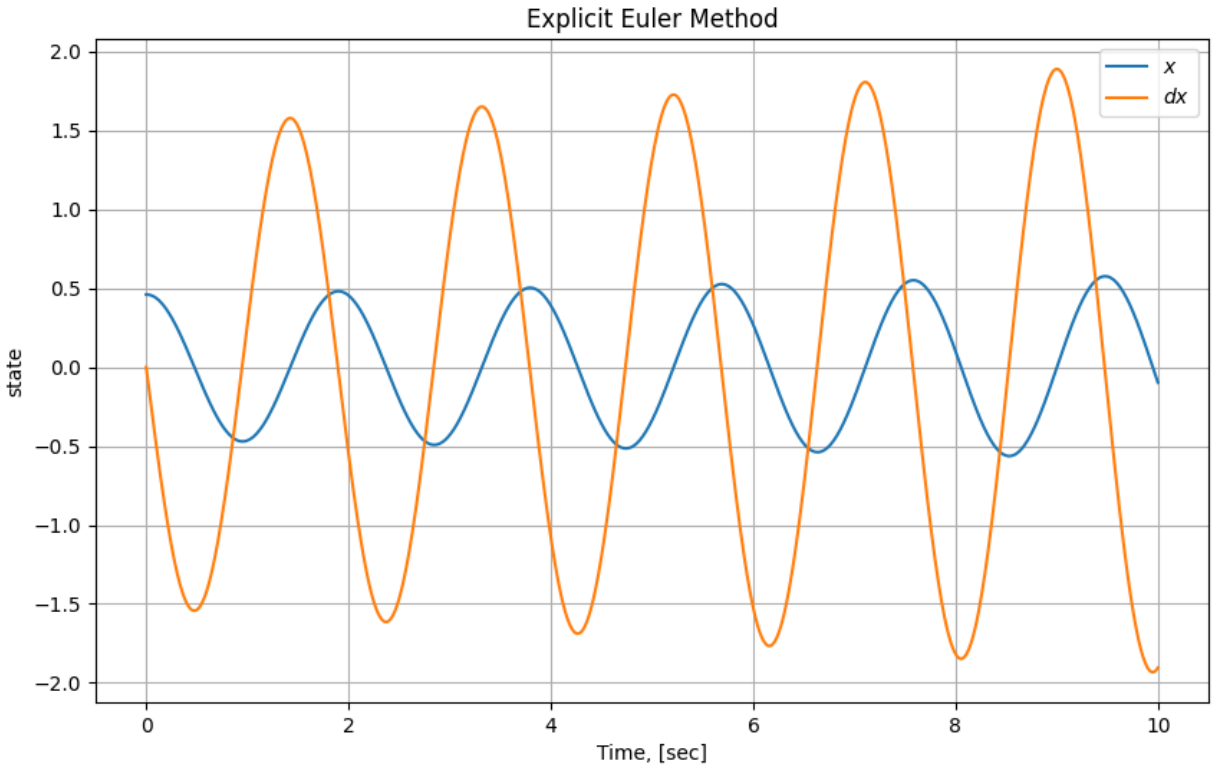


Figure 4: Relationship between state and time using Explicit Euler Integration

Looking at the graph, we can see that the system is not stable since both position and velocity diverge with time.

2.2.2 Implicit Euler

It is also known as the Backward Euler method, approximates the next state by using the derivative at the next time step rather than the current one.

$$\text{state}_{n+1} = \text{state}_n + \Delta t \cdot \text{mass-spring system}(\text{state}_{n+1}) \tag{12}$$

We define the function in Google Colab and run the simulation with time step = 0.01.

```
# Backward Euler method for Mass Spring System
def backward_euler(fun:object, x0:np.ndarray, t_f:float, h:float) -> tuple:
    # Create a time array from 0 to final time t_f, with step size h
    t = np.arange(0, t_f+h, h)
    x_hist = np.zeros((len(x0), len(t)))
    x_hist[:, 0] = x0 # Set the initial state

    for k in range(len(t)-1):
        e = 1 # an initial error value for interative solution
        x_hist[:, k+1] = x_hist[:, k]
        while e > 1e-8:
            x_n = x_hist[:, k] + h*fun(x_hist[:, k+1])
            e = np.linalg.norm(x_n - x_hist[:,k+1]) # an error value
            x_hist[:, k+1] = x_n

    return x_hist, t
```

Figure 5: Defining the Implicit Euler Integrator

By running the simulation, we get the following graph illustrating the state of the system with respect to time.

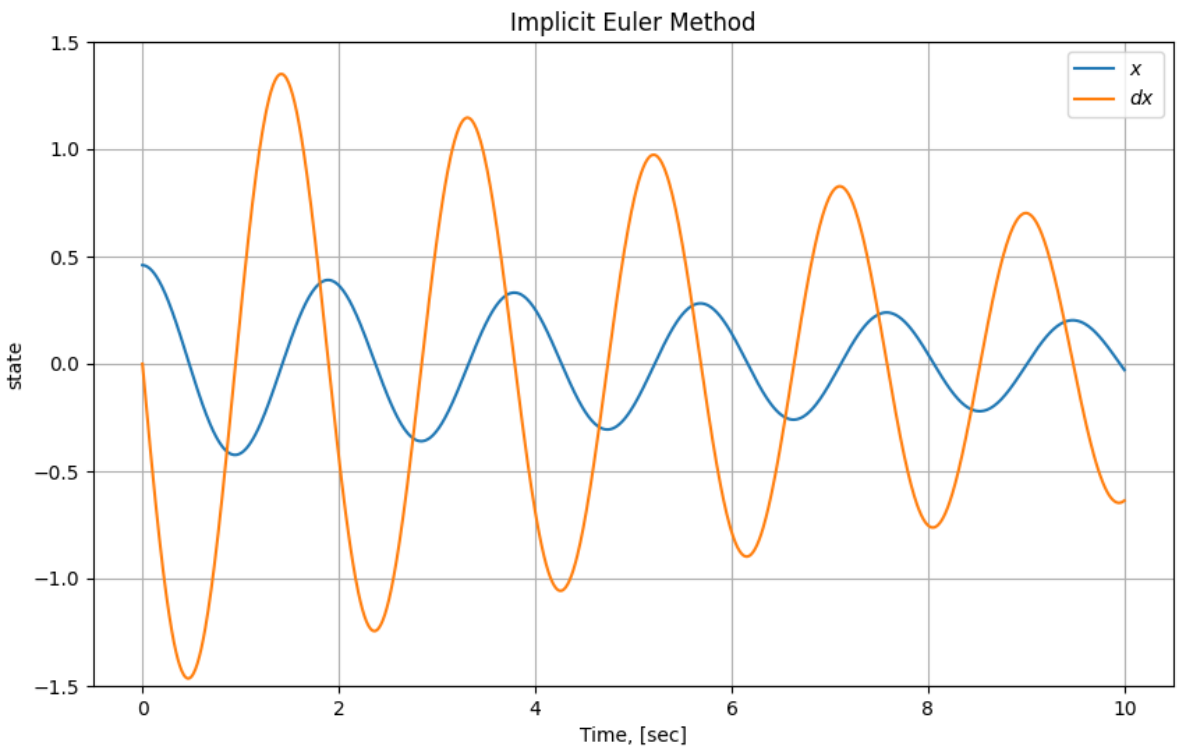


Figure 6: Relationship between states and time using Implicit Euler Integration

Looking at the graph, we can see that the system is stable since both displacement and speed converge with time.

2.2.3 Runge-Kutta of order 4 (RK4)

The 4th-order Runge-Kutta method (RK4) is a popular numerical technique for solving ordinary differential equations with high accuracy. It calculates the next state by taking a weighted average of slopes (derivatives) at several points within each time step. Specifically, it evaluates the derivative at four points: the initial state, two midpoints, and the endpoint. This approach balances computational efficiency with precision, making RK4 suitable for mass spring systems.

$$f_1 = mass_spring_system\ (state_n) \quad \textbf{(13)}$$

$$f_2 = mass_spring_system\ (state_n + h * 0.5 * f_1) \quad \textbf{(14)}$$

$$f_3 = mass_spring_system\ (state_n + h * 0.5 * f_2) \quad \textbf{(15)}$$

$$f_4 = mass_spring_system\ (state_n + h * f_3) \quad \textbf{(16)}$$

$$state_{n+1} = state_n + h/6 * (f_1 + 2f_2 + 2f_3 + f_4) \quad \textbf{(17)}$$

We define the function in Google Colab and run the simulation with time step = 0.01.

```
# Runge-Kutta 4th Order Method (RK4) function for a single step
def runge_kutta(xk, h):
    # Calculate the four Runge-Kutta increments
    f1 = mass_spring_system(xk)
    f2 = mass_spring_system(xk + 0.5*h*f1)
    f3 = mass_spring_system(xk + 0.5*h*f2)
    f4 = mass_spring_system(xk + h*f3)
    return xk + (h/6.0)*(f1 + 2*f2 + 2*f3 + f4)

# Runge-Kutta 4th Order Method for Mass Spring System
def rk4(fun: object, x0: np.ndarray, t_f: float, h: float) -> tuple:
    t = np.arange(0, t_f + h, h)
    x_hist = np.zeros((len(x0), len(t)))
    x_hist[:, 0] = x0

    for k in range(len(t) - 1):
        x_hist[:, k + 1] = runge_kutta(x_hist[:, k], h)

    return x_hist, t
```

Figure 7: Defining the RK4 Integrator

By running the simulation, we get the following graph illustrating the state of the system with respect to time.

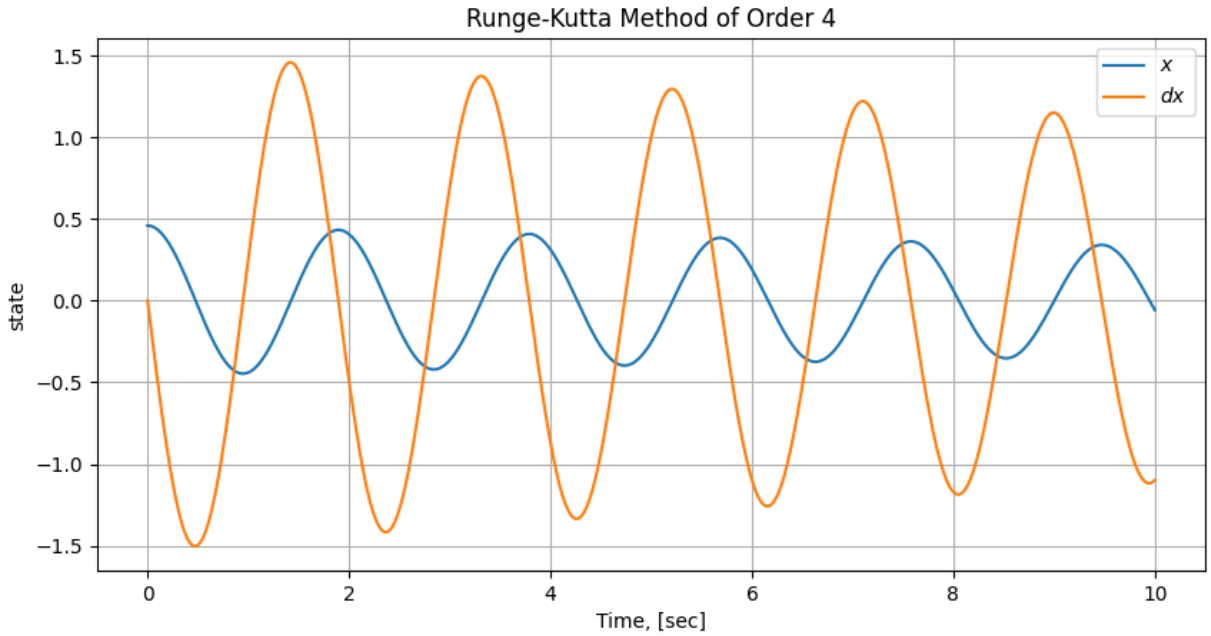


Figure 8: Relationship between state and time using RK4 Integration

Looking at the graph, we can see that the system is stable since both position and velocity converge with time.

2.3 Comparison of Results

Next, let's compare the results of the three integrators with respect to each other.

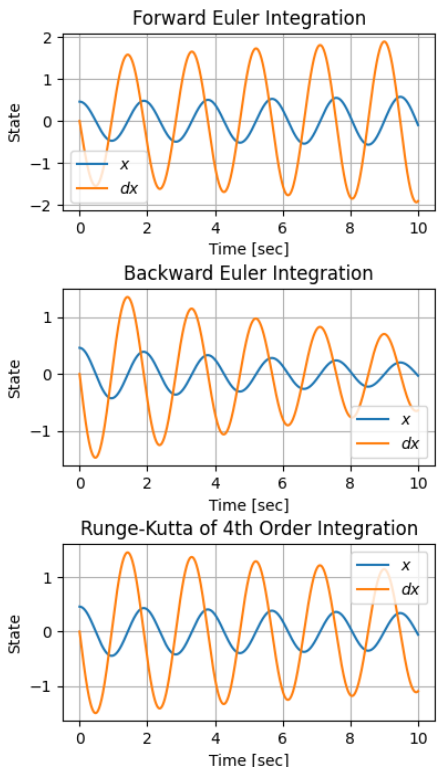


Figure 9: Relationship between state and time using all Integrators.

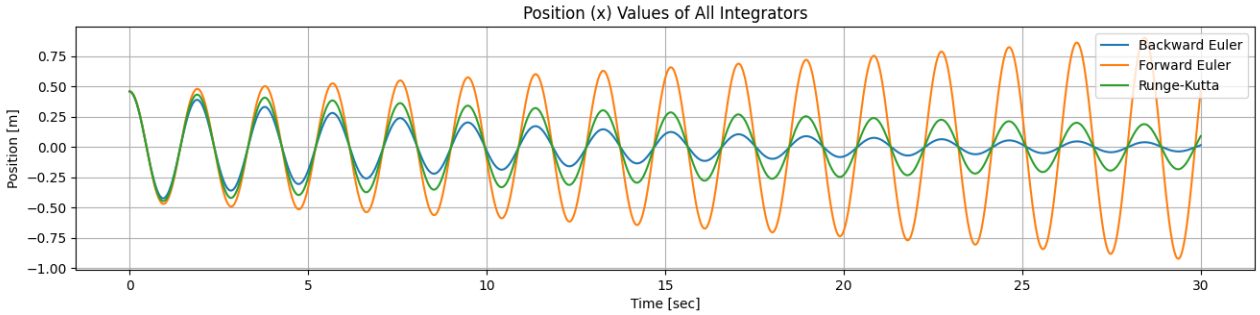


Figure 10: (a) - Graph of position (x) using each integrator with respect to time t_f = 30.

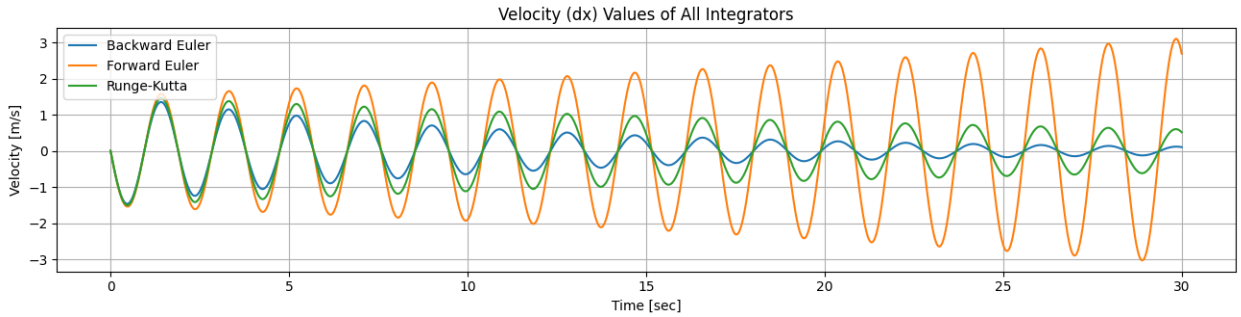


Figure 10: (b) - Graph of velocity (dx) using each integrator with respect to time t_f = 30.

2.3.1 Comparison of Integrator for Mass Spring System

Based on the results of the simulation following conclusions can be made:

- **Explicit Euler Method:** The Explicit Euler method is generally less accurate and less stable, particularly at larger time steps. By estimating the next state with the current derivative, it often accumulates error over time.

  Observed Characteristics: This method tends to "overshoot" the true solution. Values deviate from the actual oscillatory behavior, leading to artificial energy gain, and as a result, the system oscillates with increasing amplitude, eventually diverging. In the graph, sharp fluctuations become visible, particularly as the time increases, which indicates a possible decline in both accuracy and stability.

- **Implicit Euler Method:** The Implicit Euler method is more stable than the Explicit Euler method. By using the derivative at the next state, it tends to control oscillations more effectively, making it suitable for stiff systems.

  Observed Characteristics: The Implicit Euler method generally causes oscillations to decay faster, as observed in the graph. While this method helps

stabilize the system, it slightly reduces the accuracy of oscillation amplitudes. The results appear more dampened compared to the Explicit Euler method, showing smoother and more controlled oscillations. Compared to Explicit Euler, the results are smoother and more stable, though slightly less accurate than those from the RK4 method.

- **Runge-Kutta of Order 4 (RK4):** The RK4 method is both highly accurate and stable, effectively modeling the system's dynamics. It captures the mass-spring system's oscillatory behavior and damping with high precision.

  Observed Characteristics: The RK4 method closely follow the true trajectory of the system, with well-maintained oscillations that match the expected dampening and amplitude. This method avoids the artificial damping seen in Implicit Euler and the instability of Explicit Euler, providing a smooth, accurate path with minimal error over time. The method remains stable at large integration steps and time, which makes it the preferred choice.

To sum up, the Runge-Kutta of 4th order method and Implicit Euler method are more suitable for solving problems that require high accuracy, robustness and stability.

3 CONCLUSION

In this task, we were able to derive the ordinary differential equation of mass-spring system and compose the equation of motion. We were also able to implement three different integrators (Explicit Euler, Implicit Euler, Runge-Kutta of order 4) and simulate the system. By comparing the results of the simulations, we deduced that RK4 provides the highest accuracy in capturing the system's behavior, while Implicit Euler sacrifices some accuracy for stability. Explicit Euler, often fails to track the system accurately over extended periods because of its instability.